

Filter

- 🔗 Quick Start
- ❓ FAQ
- 💬 Chatting
- ⚙️ Common Settings
- 🔗 API Connections
- 📄 Prompts
- 🔍 Macros
- 🌐 World Info
- ⚙️ User Settings
- 👤 Personas
- 👤 Characters
- 👤 Welcome Page Assistants
- 🔗 Extensions
 - Blip
 - Character Expressions
 - Chat Translation
 - Chat Vectorization
 - Dynamic Audio
 - EmulatorJS
 - Image Captioning
 - Image Generation
 - Live2D
 - Objective
 - Regex
 - Retrieval-based Voice Conversion (RVC)
 - Speech Recognition
 - Summarize
 - TTS
 - AllTalk TTS V2
 - MiniMax TTS
 - XTTS with voice cloning
 - VRM
 - Web Search
 - Extras
- 🔗 Development and Automation
 - 📄 STscript Language Reference
 - 🔗 Function Calling
 - 📄 UI Extensions
 - 📄 Server Plugins
 - 🌐 Internationalization (i18n)
- ⚙️ Administration
 - ⚙️ Configuration File
 - 👤 Multi-user mode
 - 🔗 Single Sign-On (SSO)
 - 🔗 Remote connections
 - 🔗 VPNs and Tunneling
 - 📄 Reverse proxying
- 📄 License and credits

Powered by  RETYPE

UI Extensions

Copy

UI extensions expand SillyTavern's functionality by hooking into its events and API. They run in a browser context and have practically unrestricted access to the DOM, JavaScript APIs, and the SillyTavern context. Extensions can modify the UI, call internal APIs, and interact with chat data. This guide explains how to create your own extensions (JavaScript knowledge is required).

Just want to install extensions?

Go here: [Extensions](#).

To extend the functionality of the Node.js server, see the [Server Plugins](#) page.

Can't write JavaScript?

- Consider [STscript](#) as a simpler alternative to writing a full-fledged extension.
- Go through the [MDN Course](#) and come back when you're done.

Extension submissions

Want to contribute your extensions to the [official content repository](#)? Contact us!

To ensure that all extensions are safe and easy to use, we have a few requirements:

- Your extension must be open-source and have a libre license (see [Choose a License](#)). If you are unsure, AGPLv3 is a good choice.
- Extensions must be compatible with the latest release version of SillyTavern. Please be ready to update your extension if something in the core changes.
- Extensions must be well-documented. This includes a README file with installation instructions, usage examples, and a list of features.
- Extensions that have a server plugin requirement to function will not be accepted.

Examples

See live examples of simple SillyTavern extensions:

- <https://github.com/city-unit/st-extension-example> - basic extension template. Showcases manifest creation, local script imports, adding a settings UI panel, and persistent extension settings usage.
- <https://github.com/search?q=topic%3Aextension+org%3ASillyTavern&type=Repositories> - a list of all official SillyTavern extensions on GitHub.

Bundling

Extensions can also utilize bundling to isolate themselves from the rest of the modules and use any dependencies from NPM, including UI frameworks like Vue, React, etc.

- <https://github.com/SillyTavern/Extension-WebpackTemplate> - template repository of an extension using TypeScript and Webpack (no React).
- <https://github.com/SillyTavern/Extension-ReactTemplate> - template repository of a bare-bones extension using React and Webpack.

To use relative imports from the bundle, you may need to create an import wrapper. Here's an example for Webpack:

```
/**
 * Import a member from a module by URL, bypassing webpack.
 * @param (string) url URL to import from
 * @param (string) what Name of the member to import
 * @param (any) defaultValue Fallback value
 * @returns (Promise<any>) Imported member
 */
export async function importFromUrl(url, what, defaultValue = null) {
  try {
    const module = await import(/* webpackIgnore: true */ url);
    if (!Object.hasOwn(module, what)) {
      throw new Error(`No ${what} in module`);
    }
    return module[what];
  } catch (error) {
    console.error(`Failed to import ${what} from ${url}: ${error}`);
    return defaultValue;
  }
}

// Import a function from 'script.js' module
const generateRaw = await importFromUrl('/script.js', 'generateRaw');
```

manifest.json

CONTENTS

- Extension submissions
- Examples
- Bundling
- manifest.json
 - Manifest fields
 - Dependencies
 - Deprecated fields
- Scripting
 - Best practices for extension initialization
 - Using getContext
 - Shared libraries
 - TypeScript notice
 - HTML templates
 - Importing from other files
- State management
 - Persistent settings
 - Chat metadata
 - Character cards
 - Settings presets
- Internationalization
 - Direct addLocaleData call
 - Via the extension manifest
- Registering slash commands (new way)
- Events
 - Listening to events
 - Emitting events
- Prompt Interceptors
 - Registering an Interceptor
 - Interceptor Function
- Lifecycle Hooks
 - Available hooks
 - Manifest configuration
 - Implementation
- Generating text
 - Within a chat context
 - Raw generation
 - Structured Outputs
- Registering custom macros
 - New macro system
 - Legacy macro system (deprecated)
- Message formatting hooks
 - Pipeline stages
 - Registering a hook
 - Hook context
 - Hook ordering
 - Error handling
 - Full pipeline order
- Function tool calling
- Action loader
 - Options
 - Stacking loaders
 - Non-blocking loader
- Popups and user feedback
 - Popup helpers
 - Custom popups
 - Popup types

Every extension must have a folder in `data/<user-handle>/extensions` and a `manifest.json` file, which contains metadata about the extension and a path to a JS script file that is the entry point of the extension.

Downloadable extensions are mounted into the `/scripts/extensions/third-party` folder when serving over HTTP, so relative imports should be used based on that. To ease local development, consider placing your extension repository in the `/scripts/extensions/third-party` folder (the "Install for all users" option).

```
{
  "display_name": "The name of the extension",
  "loading_order": 1,
  "requires": [],
  "optional": [],
  "dependencies": [],
  "js": "index.js",
  "css": "style.css",
  "author": "Your name",
  "version": "1.0.0",
  "homePage": "https://github.com/your/extension",
  "auto_update": true,
  "minimum_client_version": "1.0.0",
  "i18n": {
    "de-de": "i18n/de-de.json"
  },
  "hooks": {
    "install": "onInstall",
    "update": "onUpdate",
    "delete": "onDelete",
    "enable": "onEnable",
    "disable": "onDisable",
    "activate": "onActivate"
  }
}
```

Manifest fields

- `display_name` is required. It is displayed in the "Manage Extensions" menu.
- `loading_order` is optional. A higher number loads later.
- `js` is the main JS file reference and is required.
- `css` is an optional style file reference.
- `author` is required. It should contain the name or contact info of the author(s).
- `auto_update` is set to `true` if the extension should auto-update when the version of the ST package changes.
- `i18n` is an optional object that specifies the supported locales and their corresponding JSON files (see below).
- `dependencies` is an optional array of strings specifying other **extensions** that this extension depends on.
- `generate_interceptor` is an optional string that specifies the name of a global function called on text generation requests.
- `minimum_client_version` is an optional string that specifies the minimum SillyTavern version required for this extension to work.
- `hooks` is an optional object that specifies **lifecycle hook** function names exported from the JS entry point module.

Dependencies

Extensions can also depend on other SillyTavern extensions. The extension will not load if any of these dependencies are missing or disabled.

Dependencies are specified by their **folder name** as it appears in the `public/extensions` directory.

Examples:

- Built-in extensions: `"vectors"`, `"caption"`
- Third-party extensions: `"third-party/Extension-WebLLM"`, `"third-party/Extension-Mermaid"`

Deprecated fields

- `requires` is an optional array of strings specifying required **Extras modules**. The extension won't be loaded if the connected Extras API doesn't provide all listed modules.
- `optional` is an optional array of strings specifying optional **Extras modules**. The extension will still load if these are missing, and the extension should handle their absence gracefully.

To check which modules are currently provided by the connected Extras API, import the `modules` array from `scripts/extensions.js`.

Scripting

Best practices for extension initialization

- Use the `activate` hook for synchronous setup that needs to run during SillyTavern's loading phase while the blocking loader is active.
- Use the `APP_INITIALIZED` event for setup that should run after all extensions and UI elements are loaded and set up, but while the loader is still blocking.
- Use the `APP_READY` event for asynchronous setup that doesn't need to block SillyTavern from being ready to use. It should use a timer or similar

mechanism to defer handling, as the event handler will be awaited.

Using `getContext`

The `getContext()` function in the `SillyTavern` global object gives you access to the SillyTavern context, which is a collection of all the main app state objects, useful functions, and utilities.

```
const context = SillyTavern.getContext();
context.chat; // Chat log - MUTABLE
context.characters; // Character list
context.characterId; // Index of the current character
context.groups; // Group list
context.groupId; // ID of the current group
// And many more...
```

You can find the full list of available properties and functions in the [SillyTavern source code](#).

🔔 If you're missing any of the functions/properties in `getContext`, please get in touch with the developers or send us a pull request!

Shared libraries

Most of the npm libraries used internally by the SillyTavern frontend are shared in the `libs` property of the `SillyTavern` global object.

- `lodash` - Utility library. [Docs](#).
- `Fuse` - Fuzzy search library. [Docs](#).
- `DOMPurify` - HTML sanitization library. [Docs](#).
- `hljs` - Syntax highlighting library. [Docs](#).
- `localforage` - Browser storage library (IndexedDB/localStorage abstraction). [Docs](#).
- `Handlebars` - Templating library. [Docs](#).
- `css` - CSS parsing/stringification tools. [Docs](#).
- `Bowser` - Browser/platform detection library. [Docs](#).
- `DiffMatchPatch` - Text diff, match, and patch library. [Docs](#).
- `Readability` / `isProbablyReadable` - Mozilla's article extraction library. [Docs](#).
- `SVGInject` - Inline SVG injection library. [Docs](#).
- `showdown` - Markdown converter library. [Docs](#).
- `moment` - Date/time manipulation library. [Docs](#).
- `seedrandom` - Seeded random number generator. [Docs](#).
- `Popper` - Tooltip/popover positioning engine. [Docs](#).
- `droll` - Dice rolling library. [Docs](#).
- `morphdom` - Fast DOM diffing/patching library. [Docs](#).
- `slideToggle` - Vanilla JS slide toggle animation. [Docs](#).
- `chalk` - Terminal string styling (limited use in browser). [Docs](#).
- `yaml` - YAML parser and stringifier. [Docs](#).
- `chevrotain` - Parser building toolkit. [Docs](#).
- `gzipSync` / `gzip` - Fast compression utilities from fflate. [Docs](#).

You can find the full list of exported libraries in the [SillyTavern source code](#).

Example: Using the DOMPurify library.

```
const { DOMPurify } = SillyTavern.libs;

const sanitizedHtml = DOMPurify.sanitize('<script>dirty HTML</script>');
```

TypeScript notice

If you want access to autocomplete for all methods in the `SillyTavern` global object (and you probably do), including `getContext()` and `libs`, you should add a TypeScript `.d.ts` module declaration. This declaration should import global types from SillyTavern's source, depending on your extension's location. Below is an example that works for both installation types: "all users" and "current user."

global.d.ts - place this file in the root of your extension directory (next to `manifest.json`):

```
export {};

// 1. Import for user-scoped extensions
import '../../../../public/global';
// 2. Import for server-scoped extensions
import '../../../../global';

// Define additional types if needed...
declare global {
  // Add global type declarations here
}
```

HTML templates

Extensions can use Handlebars HTML templates to build their UI. Place `.html` template files in your extension's directory and render them using the `renderExtensionTemplateAsync()` function from `getContext()`.

The function takes your extension's folder name, the template file name (without

.html), and an optional data object for Handlebars template variables. The returned HTML is automatically sanitized with DOMPurify and localized with `data-i18n` attributes.

```
const { renderExtensionTemplateAsync } = SillyTavern.getContext();

// Renders 'third-party/my-extension/settings.html' with the given data
const settingsHtml = await renderExtensionTemplateAsync(
  'third-party/my-extension',
  'settings',
  { title: 'My Extension', version: '1.0', defaultValue: 'test' }
);

// Append to the extensions settings panel
$('#extensions_settings2').append(settingsHtml);
```

Template file example (settings.html):

```
<div class="my-extension-settings">
  <div class="inline-drawer">
    <div class="inline-drawer-toggle inline-drawer-header">
      <b data-i18n="{{title}}">{{title}}</b>
      <div class="inline-drawer-icon fa-solid fa-circle-chevron-down
down"></div>
    </div>
    <div class="inline-drawer-content">
      <label for="my_ext_option">
        <span data-i18n="Option">Option</span>
      </label>
      <input id="my_ext_option" type="text" value="{{defaultValue}}" />
    </div>
  </div>
</div>
```

 `renderExtensionTemplate()` (synchronous) is deprecated. Always use `renderExtensionTemplateAsync()` instead.

Importing from other files

 Using imports from SillyTavern code is unreliable and can break at any time if the internal structure of ST's modules changes. `getContext` provides a more stable API.

Unless you're building a bundled extension, you can import variables and functions from other JS files.

For example, this code snippet will generate a reply from the currently selected API in the background:

```
import { generateQuietPrompt } from "../../../../../script.js";

async function handleMessage(data) {
  const text = data.message;
  const translated = await generateQuietPrompt({ quietPrompt: text });
  // ...
}
```

State management

Persistent settings

When an extension needs to persist its state, it can use the `extensionSettings` object from the `getContext()` function to store and retrieve data. An extension can store any JSON-serializable data in the settings object and must use a unique key to avoid conflicts with other extensions.

To persist the settings, use the `saveSettingsDebounce()` function, which will save the settings to the server.

```
const { extensionSettings, saveSettingsDebounce } = SillyTavern.getContext();

// Define a unique identifier for your extension
const MODULE_NAME = 'my_extension';

// Define default settings
const defaultSettings = Object.freeze({
  enabled: false,
  option1: 'default',
  option2: 5
});

// Define a function to get or initialize settings
function getSettings() {
  // Initialize settings if they don't exist
  if (!extensionSettings[MODULE_NAME]) {
    extensionSettings[MODULE_NAME] = structuredClone(defaultSettings);
  }

  // Ensure all default keys exist (helpful after updates)
  for (const key of Object.keys(defaultSettings)) {
    if (!Object.hasOwn(extensionSettings[MODULE_NAME], key)) {
      extensionSettings[MODULE_NAME][key] = defaultSettings[key];
    }
  }

  return extensionSettings[MODULE_NAME];
}
```

```
// Use the settings
const settings = getSettings();
settings.option1 = 'new value';

// Save the settings
saveSettingsDebounce();
```

Chat metadata

To bind some data to a specific chat, you can use the `chatMetadata` object from the `getContext()` function. This object allows you to store arbitrary data associated with a chat, which can be useful for storing extension-specific state.

To persist the metadata, use the `saveMetadata()` function, which will save the metadata to the server.

⚠ Do not save the reference to `chatMetadata` in a long-lived variable, as the reference will change when the chat is switched. Always use `SillyTavern.getContext().chatMetadata` to access the current chat metadata.

```
const { chatMetadata, saveMetadata } = SillyTavern.getContext();

// Set some metadata for the current chat
chatMetadata['my_key'] = 'my_value';

// Get the metadata for the current chat
const value = chatMetadata['my_key'];

// Save the metadata to the server
await saveMetadata();
```

💡 The `CHAT_CHANGED` event is emitted when the chat is switched, so you can listen to this event to update your extension's state accordingly. See more in the [Listening to events](#) section.

Character cards

SillyTavern fully supports [Character Cards V2 Specification](#), which allows to store arbitrary data in the character card JSON data.

This is useful for extensions that need to store additional data associated with the character and make it shareable when exporting the character card.

To write data to the character card `extensions` data field, use the `writeExtensionField` function from the `getContext()` function. This function takes a character ID, a string key, and a value to write. The value must be JSON-serializable.

⚠ **Weirdness Ahead**
Despite being called `characterId`, it's not a "real" unique identifier but rather an index of the character in the `characters` array.

The index of the current character is provided by the `characterId` property in the context. If you want to write data to the currently selected character, use `SillyTavern.getContext().characterId`. If you need to store data for another character, find the index by searching for the character in the `characters` array.

Caution: `characterId` is `undefined` in group chats or when no character is selected!

```
const { writeExtensionField, characterId } = SillyTavern.getContext();

// Write some data to the character card
await writeExtensionField(characterId, 'my_extension_key', {
  someData: 'value',
  anotherData: 42
});

// Read the data back from the character card
const character = SillyTavern.getContext().characters[characterId];
// The data is stored in the 'extensions' object of the character's data
const myData = character.data?.extensions?.my_extension_key;
```

Settings presets

Arbitrary JSON data can be stored in the settings presets of the main API types. It will be exported and imported along with the preset JSON, so you can use it to store extension-specific settings for the preset. The following API types support data extensions in settings presets:

- Chat Completion
- Text Completion
- NovelAI
- KoboldAI / AI Horde

To read or write the data, you first need to get the `PresetManager` instance from the context:

```
const { getPresetManager } = SillyTavern.getContext();

// Get the preset manager for the current API type
const pm = getPresetManager();

// Write data to the preset extension field:
// - path: the path to the field in the preset data
// - value: the value to write
// - name (optional): the name of the preset to write to, defaults to the
// currently selected preset
await pm.writePresetExtensionField({ path: 'hello', value: 'world' });

// Read data from the preset extension field:
// - path: the path to the field in the preset data
// - name (optional): the name of the preset to read from, defaults to the
// currently selected preset
const value = pm.readPresetExtensionField({ path: 'hello' });
```

💡 The `PRESET_CHANGED` and `MAIN_API_CHANGED` events are emitted when the preset is changed or the main API is switched, so you can listen to these events to update your extension's state accordingly. See more in the [Listening to events](#) section.

Internationalization

📘 For general information on providing translations, see the [Internationalization](#) page.

Extensions can provide additional localized strings for use with the `t`, `translate` functions and the `data-i18n` attribute in HTML templates.

See the list of supported locales here (`lang` key):
<https://github.com/SillyTavern/SillyTavern/blob/release/public/locales/lang.json>

Direct `addLocaleData` call

Pass a locale code and an object with the translations to the `addLocaleData` function. Overrides of existing keys are *NOT* allowed. If the passed locale code is not a currently chosen locale, the data will be silently ignored.

```
SillyTavern.getContext().addLocaleData('fr-fr', { 'Hello': 'Bonjour' });
SillyTavern.getContext().addLocaleData('de-de', { 'Hello': 'Hallo' });
```

Via the extension manifest

Add an `i18n` object with a list of supported locales and their corresponding JSON file paths (relative to your extension's directory) to the manifest.

```
{
  "display_name": "Foobar",
  "js": "index.js",
  // rest of the fields
  "i18n": {
    "fr-fr": "i18n/french.json",
    "de-de": "i18n/german.json"
  }
}
```

Registering slash commands (new way)

While `registerSlashCommand` still exists for backward compatibility, new slash commands should now be registered through `SlashCommandParser.addCommandObject()` to provide extended details about the command and its parameters to the parser (and, in turn, to autocomplete and the command help).

```
SlashCommandParser.addCommandObject(SlashCommand.fromProps({ name: 'repeat',
  callback: (namedArgs, unnamedArgs) => {
    return Array(namedArgs.times ?? 5)
      .fill(unnamedArgs.toString())
      .join(isTrueBoolean(namedArgs.space.toString()) ? ' ' : '');
  },
  aliases: ['example-command'],
  returns: 'the repeated text',
  namedArgumentList: [
    SlashCommandNamedArgument.fromProps({ name: 'times',
      description: 'number of times to repeat the text',
      typeList: ARGUMENT_TYPE.NUMBER,
      defaultValue: '5',
    }),
    SlashCommandNamedArgument.fromProps({ name: 'space',
      description: 'whether to separate the texts with a space',
      typeList: ARGUMENT_TYPE.BOOLEAN,
      defaultValue: 'off',
      enumList: ['on', 'off'],
    }),
  ],
  unnamedArgumentList: [
    SlashCommandArgument.fromProps({ description: 'the text to repeat',
      typeList: ARGUMENT_TYPE.STRING,
      isRequired: true,
    }),
  ],
}));
```

```

    helpString: `
      <div>
        Repeats the provided text a number of times.
      </div>
      <div>
        <strong>Example:</strong>
        <ul>
          <li>
            <pre><code class="language-stscript">/repeat foo</code>

          </li>
          <li>
            <pre><code class="language-stscript">/repeat times=3
space=on bar</code></pre>
            returns "bar bar bar"
          </li>
        </ul>
      </div>
    `;
  });
});

```

All registered commands can be used in **STscript** in any possible way.

Events

Listening to events

Use `eventSource.on(eventType, eventHandler)` to listen for events:

```

const { eventSource, event_types } = SillyTavern.getContext();

eventSource.on(event_types.MESSAGE_RECEIVED, handleIncomingMessage);

function handleIncomingMessage(data) {
  // Handle message
}

```

The main event types are:

App lifecycle:

- **APP_INITIALIZED** : the app is initialized and close to being ready, but the loader is still visible. UI modifications can be done here. It will auto-fire every time a new listener is attached after the app is initialized.
- **APP_READY** : the app is fully loaded and ready to use. It will auto-fire every time a new listener is attached after the app is ready.

Messages:

- **MESSAGE_SENT** : the message is sent by the user and recorded into the `chat` object but not yet rendered in the UI.
- **MESSAGE_RECEIVED** : the LLM message is generated and recorded into the `chat` object but not yet rendered in the UI.
- **USER_MESSAGE_RENDERED** : the message sent by a user is rendered in the UI.
- **CHARACTER_MESSAGE_RENDERED** : the generated LLM message is rendered in the UI.
- **MESSAGE_EDITED** : a message has been edited by the user.
- **MESSAGE_DELETED** : a message has been deleted.
- **MESSAGE_SWIPED** : a message swipe has been triggered.
- **STREAM_TOKEN_RECEIVED** : a new token was received during streaming generation.

Generation:

- **GENERATION_AFTER_COMMANDS** : the generation is about to start after processing slash commands.
- **GENERATION_STARTED** : the generation has started.
- **GENERATION_STOPPED** : the generation was stopped by the user.
- **GENERATION_ENDED** : the generation has been completed or has errored out.

Chat:

- **CHAT_CHANGED** : the chat has been switched (e.g., switched to another character, or another chat was loaded).
- **CHAT_CREATED** : a new chat has been created.
- **CHAT_DELETED** : a chat has been deleted.

Characters:

- **CHARACTER_EDITED** : a character's data has been changed.
- **CHARACTER_DELETED** : a character has been deleted.
- **CHARACTER_DUPLICATED** : a character has been duplicated.

Persona:

- **PERSONA_CHANGED** : the active persona has been changed.
- **PERSONA_CREATED** : a new persona has been created.
- **PERSONA_UPDATED** : a persona has been updated.
- **PERSONA_RENAMED** : a persona has been renamed.
- **PERSONA_DELETED** : a persona has been deleted.

Settings and presets:

- **SETTINGS_UPDATED** : the application settings have been updated.
- **PRESET_CHANGED** : the active preset has been changed.
- **MAIN_API_CHANGED** : the main API type has been switched.
- **CHATCOMPLETION_SOURCE_CHANGED** : the chat completion source has changed.

- `CHATCOMPLETION_MODEL_CHANGED` : the chat completion model has changed.
- `CONNECTION_PROFILE_LOADED` : a connection profile has been loaded.

World Info:

- `WORLDINFO_UPDATED` : world info data has been updated.
- `WORLDINFO_SETTINGS_UPDATED` : world info settings have been changed.

Tool calling:

- `TOOL_CALLS_PERFORMED` : tool calls have been executed.
- `TOOL_CALLS_RENDERED` : tool call results have been rendered in the chat.

Text-to-Speech:

- `TTS_JOB_STARTED` : a TTS job has started.
- `TTS_AUDIO_READY` : TTS audio data is ready to be played.
- `TTS_JOB_COMPLETE` : a TTS job has been completed.

The full list of event types can be found [in the source](#).

Event data

The way each event passes its data to the listener is not uniform. Some events don't emit any data; some pass an object or a primitive value. Please refer to the source code where the event is emitted to see what data it passes, or check with the debugger.

Emitting events

You can produce any application events from extensions, including custom events, by calling `eventSource.emit(eventType, ...eventData)` :

```
const { eventSource } = SillyTavern.getContext();

// Can be a built-in event_types field or any string.
const eventType = 'myCustomEvent';

// Use 'await' to ensure all event handlers complete before continuing
// execution.
await eventSource.emit(eventType, { data: 'custom event data' });
```

Prompt Interceptors

Prompt Interceptors provide a way for extensions to perform any activity such as modifying the chat data, adding injections, or aborting the generation before a text generation request is made.

Interceptors from different extensions are run sequentially. The order is determined by the `loading_order` field in their respective `manifest.json` files. Extensions with lower `loading_order` values run earlier. If `loading_order` is not specified, `display_name` is used as a fallback. If neither is specified, the order is undefined.

Registering an Interceptor

To define a prompt interceptor, add a `generate_interceptor` field to your extension's `manifest.json` file. The value should be the name of a global function that will be called by SillyTavern.

```
{
  "display_name": "My Interceptor Extension",
  "loading_order": 10, // Affects execution order
  "generate_interceptor": "myCustomInterceptorFunction",
  // ... other manifest properties
}
```

Interceptor Function

The `generate_interceptor` function is a global function that will be called upon generation requests that are not dry runs. It must be defined in the global scope (e.g., `globalThis.myCustomInterceptorFunction = async function(...) { ... }`) and can return a `Promise` if it needs to perform any asynchronous operations.

The interceptor function receives the following arguments:

- `chat` : An array of message objects representing the chat history that will be used for prompt building. You can modify this array directly (e.g., add, remove, or alter messages). Please note that messages are mutable, so any changes you make to the array will be reflected in the actual chat history. If you want the changes to be ephemeral, use `structuredClone` to create a deep copy of the message object.
- `contextSize` : A number indicating the current context size (in tokens) calculated for the upcoming generation.
- `abort` : A function that, when called, will signal to prevent the text generation from proceeding. It accepts a boolean parameter that prevents any subsequent interceptors from running if `true`.
- `type` : A string indicating the type or trigger of the generation (e.g., `'quiet'`, `'regenerate'`, `'impersonate'`, `'swipe'`, etc.). This helps the interceptor apply logic conditionally based on how the generation was initiated.

Example Implementation:


```
globalThis.myCustomInterceptorFunction = async function(chat, contextSize, abort, type) {
  // Example: Add a system note before the last user message
  const systemNote = {
    is_user: false,
    name: "System Note",
    send_date: Date.now(),
    mes: "This was added by my extension!"
  };
  // Insert before the last message
  chat.splice(chat.length - 1, 0, systemNote);
}
```

Lifecycle Hooks

Extensions can define lifecycle hooks in `manifest.json` that are called at specific points in the extension's lifecycle. Each hook maps to an **exported function** from the extension's JS entry point module (the file specified in the `js` field).

All hooks are optional. Hook functions can return a `Promise` that will be awaited (with a 5-second timeout). If a hook exceeds the timeout, a warning is logged and execution continues. Errors in hooks are caught and logged without blocking the operation.

Available hooks

Hook	When it's called
<code>activate</code>	When the extension is successfully activated during page load
<code>install</code>	After the extension is installed and its settings are loaded
<code>update</code>	After a successful extension update (before the reload toast)
<code>delete</code>	Before the extension is deleted from the server
<code>enable</code>	Before the extension is enabled and settings are saved
<code>disable</code>	Before the extension is disabled and settings are saved
<code>clean</code>	When a user clicks the "Clean extension data" button in the extension manager, or chooses an option to clean when deleting the extension

Manifest configuration

Add a `hooks` object to your `manifest.json` mapping hook names to exported function names:

```
{
  "display_name": "My Extension",
  "js": "index.js",
  // Other fields here...
  "hooks": {
    "install": "onInstall",
    "update": "onUpdate",
    "delete": "onDelete",
    "enable": "onEnable",
    "disable": "onDisable",
    "activate": "onActivate",
    "clean": "onClean"
  }
}
```

The names can be freely chosen, as long as the are valid JS function names. Any number of hooks can be configured, you do not have to fill out and implement all of them.

Implementation

Export the hook functions from your main JS entry point. Each function receives no arguments and can optionally return a `Promise`:

```
// index.js - your extension's entry point

export async function onInstall() {
  console.log('Extension installed! Performing first-time setup...');
  // e.g., initialize default data, create storage entries
}

export async function onActivate() {
  console.log('Extension activated during page load');
}

export async function onUpdate() {
  console.log('Extension updated! Running migrations...');
  // e.g., migrate data from old format to new format
}

export async function onDelete() {
  console.log('Extension about to be deleted. Cleaning up...');
  // e.g., remove stored data, clean up localStorage
  const { localStorage } = SillyTavern.libs;
  await localStorage.removeItem('my_extension_data');
```

```

}

export function onEnable() {
  console.log('Extension enabled');
}

export function onDisable() {
  console.log('Extension disabled');
}

export async function onClean() {
  console.log('Extension data cleaned');
  // e.g., cleanup of the extension's data here
}

```

⚠ Hook functions have a **5-second timeout**. If your hook takes longer, execution will continue and a warning will be logged. Keep hook logic fast and lightweight.

Generating text

SillyTavern provides several functions to generate text in different contexts using the currently chosen LLM API. These functions allow you to generate text in the context of a chat, raw generation without any context, or with structured outputs.

Within a chat context

The `generateQuietPrompt()` function is used to generate text in the context of a chat with an added "quiet" prompt (post-history instruction) in the background (the output is not rendered in the UI). This is useful for generating text without interrupting the user experience while also keeping the relevant chat and character data intact, such as generating a summary or an image prompt.

```

const { generateQuietPrompt } = SillyTavern.getContext();

const quietPrompt = 'Generate a summary of the chat history.';

const result = await generateQuietPrompt({
  quietPrompt,
});

```

Raw generation

The `generateRaw()` function is used to generate text without any chat context. It is useful when you want to fully control the prompt-building process.

It accepts a `prompt` as a Text Completion string or an array of Chat Completion objects, constructing the request with an appropriate format depending on the selected API type, e.g., converting between chat/text modes, applying instruct formatting, etc. You can also pass an additional `systemPrompt` and a `prefill` to the function for even more control over the generation process.

```

const { generateRaw } = SillyTavern.getContext();

const systemPrompt = 'You are a helpful assistant.';
const prompt = 'Generate a story about a brave knight.';
const prefill = 'Once upon a time,';

/*
In Chat Completion mode, will produce a prompt like this:
[
  {role: 'system', content: 'You are a helpful assistant.'},
  {role: 'user', content: 'Generate a story about a brave knight.'},
  {role: 'assistant', content: 'Once upon a time,'}
]
*/

/*
In Text Completion mode (no instruct), will produce a prompt like this:
"You are a helpful assistant.\nGenerate a story about a brave knight.\nOnce upon a time,"
*/

const result = await generateRaw({
  systemPrompt,
  prompt,
  prefill,
});

```

Structured Outputs

❗ Currently only supported by the Chat Completion API. The availability varies based on the selected source and model. If the selected model does not support structured outputs, the generation will either fail or will return an empty object (`{}`). Check the documentation for the specific API you are using to see if structured outputs are supported.

You can use the structured outputs feature to ensure the model produces a valid JSON object that adheres to a provided **JSON Schema**. This is useful for extensions that require structured data, such as state tracking, data classification, etc.

To use structured outputs, you must pass a JSON schema object to `generateRaw()` or `generateQuietPrompt()`. The model will then generate a

response that matches the schema, and it will be returned as a stringified JSON object.

⚠ The outputs are not validated against the schema, you must handle the parsing and validation of the generated output yourself. If the model fails to generate a valid JSON object, the function will return an empty object (`{}`).

[Zod](#) is a popular library to generate and validate JSON schemas. Its use will not be covered here.

```
const { generateRaw, generateQuietPrompt } = SillyTavern.getContext();

// Define a JSON schema for the expected output
const jsonSchema = {
  // Required: a name for the schema
  name: 'StoryStateModel',
  // Optional: a description of the schema
  description: 'A schema for a story state with location, plans, and memories.',
  // Optional: the schema will be used in strict mode, meaning that only the fields defined in the schema will be allowed
  strict: true,
  // Required: a definition of the schema
  value: {
    '$schema': 'http://json-schema.org/draft-04/schema#',
    'type': 'object',
    'properties': {
      'location': {
        'type': 'string'
      },
      'plans': {
        'type': 'string'
      },
      'memories': {
        'type': 'string'
      }
    },
    'required': [
      'location',
      'plans',
      'memories'
    ],
  },
};

const prompt = 'Generate a story state with location, plans, and memories. Output as a JSON object.';

const rawResult = await generateRaw({
  prompt,
  jsonSchema,
});

const quietResult = await generateQuietPrompt({
  quietPrompt: prompt,
  jsonSchema,
});
```

Registering custom macros

You can register custom macros that can be used anywhere where macro substitutions are supported, e.g. in the character card fields, STscript commands, prompt templates, etc.

New macro system

The recommended way to register macros is through the `macros.register()` function available via `SillyTavern.getContext()`. This system supports arguments, categories, descriptions, and rich documentation metadata.

```
const { macros } = SillyTavern.getContext();

// Simple macro with a handler function
macros.register('tomorrow', {
  description: 'Returns tomorrow\'s date',
  handler: () => {
    return new Date(Date.now() + 24 * 60 * 60 * 1000).toLocaleDateString();
  },
});

// Macro with unnamed arguments and a category
macros.register('greet', {
  description: 'Generates a greeting for the given name',
  category: macros.category.UTILITY,
  unnamedArgs: [
    { name: 'name', description: 'The name to greet' },
  ],
  handler: ({ unnamedArgs }) => {
    const [name] = unnamedArgs;
    return `Hello, ${name}!`;
  },
});
```

The `handler` function receives a `MacroExecutionContext` object containing

- `args` - All unnamed arguments passed to the macro.
- `unnamedArgs` - Positional arguments matching the defined argument list.
- `list` - List arguments (after unnamed args), or `null` if list is not enabled.
- `env` - The macro environment with access to character data, chat state, etc.
- `resolve(text)` - Function to resolve nested macros in text (when

```
delayArgResolution is true ).
```

And more.

Handlers will run synchronously, so they can never return a `Promise` or call async actions synchronously.

To unregister a macro:

```
const { macros } = SillyTavern.getContext();

macros.registry.unregisterMacro('greet');
```

You can also register aliases for existing macros:

```
const { macros } = SillyTavern.getContext();

macros.registerAlias('greet', 'hello', { visible: true });
```

Legacy macro system (deprecated)

⚠️ `registerMacro()` and `unregisterMacro()` from `getContext()` are **deprecated**. Use `macros.register()` and `macros.registry.unregisterMacro()` instead.

The legacy API is still available for backward compatibility, but will be removed in a future release:

```
const { registerMacro, unregisterMacro } = SillyTavern.getContext();

// Simple string macro
registerMacro('fizz', 'buzz');
// Function macro (must be synchronous)
registerMacro('tomorrow', () => {
  return new Date(Date.now() + 24 * 60 * 60 * 1000).toLocaleDateString();
});

// Unregister
unregisterMacro('fizz');
```

Message formatting hooks

⚠️ **Staging Feature**
This is currently only available on the `staging` branch of SillyTavern, and not part of the latest release.

Extensions can hook into the message formatting pipeline to transform message text before it reaches the DOM. This is useful for adding annotations (ruby tags, tooltips), highlighting, or custom text transformations.

⚠️ Hooks run synchronously and **must return a string**. Async functions and non-string returns will throw a `TypeError` at registration time or be silently ignored at runtime with a console warning. Do not perform expensive operations in these hooks — they run on every message render.

Pipeline stages

Hooks can be registered for three pipeline stages. All stages run **before** DOMPurify sanitization, so output is always safe:

Stage	When it runs	Text format
<code>beforeRegex</code>	After prompt-bias stripping, before custom regex rules	Plain text
<code>afterRegex</code>	After custom regex rules, before Markdown conversion	Plain text
<code>afterMarkdown</code>	After Markdown-to-HTML conversion (showdown), before DOMPurify	HTML string

The `afterMarkdown` stage is the default and the most common insertion point for extensions that want to annotate rendered HTML.

Registering a hook

Access the `messageFormatter` from `getContext()`:

```
const { messageFormatter } = SillyTavern.getContext();

// Simple hook - transforms message text
messageFormatter.addHook((mes, ctx) => {
  // Skip user messages
  if (ctx.isUser) return mes;
});
```

```
// Add furigana to Japanese text
return addFurigana(mes);
});

// Hook with explicit stage and order
messageFormatter.addHook((mes, ctx) => {
  // Transform after Markdown conversion but before sanitization
  return mes.replace(/\p{(?:.+)}`"/g, '<mark>$1</mark>');
}, {
  stage: messageFormatter.stage.AFTER_MARKDOWN,
  order: messageFormatter.order.EARLY,
});
```

Hook context

The hook receives an immutable context object with message metadata:

Property	Type	Description
<code>characterName</code>	<code>string</code>	Character name associated with the message
<code>isSystem</code>	<code>boolean</code>	Whether the message is a system message
<code>isUser</code>	<code>boolean</code>	Whether the message was sent by the user
<code>messageId</code>	<code>number</code>	Index of the message in the chat array, or <code>-1</code> for transient messages (streaming previews)
<code>isReasoning</code>	<code>boolean</code>	Whether the message is reasoning/thinking output
<code>stage</code>	<code>string</code>	The pipeline stage currently being executed

The context object is frozen with `Object.freeze()` — attempting to modify it will have no effect.

Hook ordering

Hooks within a stage run in ascending order. Use the `order` option to control execution order:

```
const { hook_order } = messageFormatter;

// Predefined constants
hook_order.EARLIEST; // 0
hook_order.EARLY; // 10
hook_order.NORMAL; // 50 (default)
hook_order.LATE; // 90
hook_order.LATEST; // 100

// Custom numeric value
messageFormatter.addHook(myHook, { order: 25 });
```

Lower numbers run first. This is useful when multiple extensions transform the same text — for example, one extension might extract data early, and another might format it later.

Error handling

Hook execution is wrapped in try/catch. If a hook throws, it is skipped and a console error is logged — the pipeline continues with the remaining hooks.

If a hook returns a non-string value (including `undefined` or a `Promise`), a console warning is emitted and the return value is ignored. The pipeline continues with the previous text unchanged.

Full pipeline order

For reference, the complete message formatting pipeline is:

- Prompt-bias stripping (message 0 only)
- Comment / hidden-message normalization
- `beforeRegex` extension hooks
- Custom regex rules (`getRegexString`)
- `afterRegex` extension hooks
- Markdown auto-fix (`fixMarkdown`)
- HTML tag encoding (`encode_tags`)
- Showdown Markdown → HTML conversion
- `afterMarkdown` extension hooks
- Name-prefix stripping (`allow_name2_display`)
- DOMPurify sanitization

All extension hooks (steps 3, 5, 9) run **before** DOMPurify so their output is always sanitized.

Function tool calling

Extensions can register custom function tools that the LLM can invoke during

chat completion. This lets your extension react to structured data from the model — for example, querying APIs, performing calculations, or triggering extension features.

For a full guide including prerequisites, supported APIs, registration fields, and tips, see the dedicated [Function Calling](#) page.

Quick example:

```
const { registerFunctionTool } = SillyTavern.getContext();

registerFunctionTool({
  name: 'get_weather',
  displayName: 'Get Weather',
  description: 'Get the current weather for a given location',
  parameters: {
    $schema: 'http://json-schema.org/draft-04/schema#',
    type: 'object',
    properties: {
      location: { type: 'string', description: 'City name' },
    },
    required: ['location'],
  },
  action: async ({ location }) => {
    const data = await fetchWeatherData(location);
    return JSON.stringify(data);
  },
});
```

Action loader

The action loader provides a loading overlay and toast notification system for long-running operations. It replaces the deprecated `showLoader()` / `hideLoader()` functions.

Access it via `loader` from `getContext()` :

```
const { loader } = SillyTavern.getContext();

// Basic blocking loader with a stoppable toast
const handle = loader.show({ message: 'Processing data...' });
try {
  const result = await someExpensiveOperation();
} finally {
  await handle.hide();
}
```

Options

Option	Default	Description
<code>blocking</code>	<code>true</code>	Show a full-screen overlay that blocks interaction
<code>message</code>	<code>'Generating...'</code>	Message shown in the toast notification
<code>title</code>	<code>''</code>	Optional title for the toast
<code>toastMode</code>	<code>'stoppable'</code>	<code>'stoppable'</code> (with stop button), <code>'static'</code> (no button), or <code>'none'</code> (no toast)
<code>stopTooltip</code>	<code>'Stop'</code>	Tooltip text for the stop button
<code>onStop</code>	<code>null</code>	Custom stop handler. Defaults to <code>stopGeneration()</code>
<code>onHide</code>	<code>null</code>	Called when the loader is hidden (not stopped)
<code>overlayContent</code>	<code>null</code>	Custom HTML element or string replacing the default spinner

Stacking loaders

Multiple loaders can be active simultaneously. The overlay stays visible as long as at least one blocking loader is active:

```
const { loader } = SillyTavern.getContext();

const loader1 = loader.show({ message: 'Task 1...' });
const loader2 = loader.show({ message: 'Task 2...' });
await loader1.hide(); // Overlay stays - loader2 is still active
await loader2.hide(); // Now overlay hides
```

Non-blocking loader

For background tasks that shouldn't block the UI:

```
const { loader } = SillyTavern.getContext();

const handle = loader.show({
  blocking: false,
```

```
message: 'Downloading in background...',
onStop: () => abortDownload(),
});
```

Popups and user feedback

Popup helpers

SillyTavern provides convenient popup helpers via `Popup.show` from `getContext()` :

```
const { Popup } = SillyTavern.getContext();

// Confirmation dialog - returns POPUP_RESULT.AFFIRMATIVE or
// POPUP_RESULT.NEGATIVE
const confirmed = await Popup.show.confirm('Confirm Action', 'Are you sure you
want to proceed?');

// Text input dialog - returns the entered string, or null if cancelled
const userInput = await Popup.show.input('Enter Name', 'Please provide a
name:', 'default value');

// Information display - returns the clicked button result
await Popup.show.text('Info', 'Operation completed successfully.');
```

Custom popups

For more complex popups, instantiate `Popup` directly with full options:

```
const { Popup, POPUP_TYPE, POPUP_RESULT } = SillyTavern.getContext();

const popup = new Popup(
  '<div>Custom HTML content here</div>',
  POPUP_TYPE.TEXT,
  '',
  {
    wide: true, // Wide display mode
    okButton: 'Save', // Custom OK button text
    cancelButton: 'Discard', // Custom Cancel button text
    customButtons: [
      {
        text: 'Export',
        icon: 'fa-download',
        result: POPUP_RESULT.CUSTOM1,
      },
    ],
    customInputs: [
      {
        id: 'my_checkbox',
        label: 'Enable feature',
        type: 'checkbox',
        defaultState: false,
      },
    ],
    allowVerticalScrolling: true,
  }
);

const result = await popup.show();

if (result === POPUP_RESULT.AFFIRMATIVE) {
  // OK was clicked
} else if (result === POPUP_RESULT.CUSTOM1) {
  // Export button was clicked
}

// Read custom input values
const checkboxValue = popup.inputResults?.get('my_checkbox');
```

Popup types

Type	Description
<code>POPUP_TYPE.TEXT</code>	General content popup with buttons
<code>POPUP_TYPE.CONFIRM</code>	Yes/No confirmation dialog
<code>POPUP_TYPE.INPUT</code>	Popup with a text input field
<code>POPUP_TYPE.DISPLAY</code>	Content-only popup with a close button
<code>POPUP_TYPE.CROP</code>	Image cropping popup

Toast notifications

For lightweight feedback, use `toastr` (globally available):

```
toastr.success('Data saved successfully');
toastr.error('Failed to connect to API');
toastr.warning('This feature is experimental');
toastr.info('Processing...');
```

Data bank scrapers

Extensions can register custom data scrapers for the Data Bank feature. Scrapers provide a way to import data from custom sources (e.g., web pages, APIs, file formats):

```
const { registerDataBankScraper } = SillyTavern.getContext();

await registerDataBankScraper({
  id: 'my_scraper',
  name: 'My Data Source',
  description: 'Import data from My Data Source',
  iconClass: 'fa-solid fa-database',
  iconAvailable: true,
  isAvailable: async () => true,
  scrape: async () => {
    // Return an array of File objects
    const content = await fetchDataFromSource();
    return [new File([content], 'data.txt', { type: 'text/plain' })];
  },
});
```

Debug functions

Extensions can register custom debug functions that appear in the Debug Menu (accessible via the power user settings). This is useful for exposing diagnostic tools, cache/cleanup functionality or manual triggers during development:

```
const { registerDebugFunction } = SillyTavern.getContext();

registerDebugFunction(
  'my_ext_clear_cache', // Unique function ID
  'Clear My Extension Cache', // Display name
  'Clears all cached data for My Extension', // Description
  async () => {
    const { localforage } = SillyTavern.libs;
    await localforage.removeItem('my_extension_cache');
    toastr.success('Cache cleared');
  }
);
```

Do Extras request

 The Extras API is deprecated. It's not recommended to use it in new extensions.

The `doExtrasFetch()` function allows you to make requests to your SillyTavern Extras API server.

For example, to call the `/api/summarize` endpoint:

```
import { getApiUrl, doExtrasFetch } from "../../extensions.js";

const url = new URL(getApiUrl());
url.pathname = '/api/summarize';

const apiResult = await doExtrasFetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Bypass-Tunnel-Reminder': 'bypass',
  },
  body: JSON.stringify({
    // Request body
  })
});
```

`getApiUrl()` returns the base URL of the Extras server.

The `doExtrasFetch()` function:

- Adds the `Authorization` and `Bypass-Tunnel-Reminder` headers
- Handles fetching the result
- Returns the result (the response object)

This makes it easy to call your Extras API from your extension.

You can specify:

- The request method: GET, POST, etc.
- Additional headers
- The body for POST requests
- Any other fetch options

Best Practices

Security

Never store API keys or secrets in `extensionSettings`

Extension settings are accessible to all other extensions and are stored in plain text. Do not store sensitive data client-side:

```
// BAD - Don't do this!
extensionSettings[MODULE_NAME].apiKey = 'secret_key_123';

// NOTE: There is no secure way to store secrets in client-side extensions.
// If you need to handle sensitive data, use server plugins instead.
// See: https://docs.sillytavern.app/for-contributors/server-plugins/
```

Sanitize user inputs

Always validate and sanitize data from user inputs before using it in commands, API calls, or DOM manipulation:

```
// Validate input type first
if (typeof userInput !== 'string') {
  toastr.error('Invalid input type');
  return;
}
// Use DOMPurify to sanitize HTML input
const { DOMPurify } = SillyTavern.libs;
const cleanInput = DOMPurify.sanitize(userInput);
```

Avoid using `eval()` or `Function()` constructors

These can execute arbitrary code and pose security risks. If you need dynamic evaluation, use safer alternatives or restrict the input carefully.

Performance

Don't store large data in `extensionSettings`

Extension settings are loaded into memory and saved frequently. Large data can cause performance issues:

```
// BAD - Don't store large data
extensionSettings[MODULE_NAME].largeDataset = { /* megabytes of data */ };

// GOOD - Use localforage (abstraction over IndexedDB/localStorage)
const { localforage } = SillyTavern.libs;
await localforage.setItem(`${MODULE_NAME}_data`, largeData);

// Or use localStorage for smaller data
localStorage.setItem(`${MODULE_NAME}_data`, JSON.stringify(smallData));
```

Clean up event listeners

Remove event listeners when they're no longer needed to prevent memory leaks:

```
function cleanup() {
  eventSource.removeListener(event_types.MESSAGE_RECEIVED, handleMessage);
  document.getElementById('myElement').removeEventListener('click', handleClick);
}
```

Don't block the UI thread

For heavy operations, use `async/await` or web workers:

```
// Use async for I/O operations
async function processData() {
  const result = await fetch('/api/process');
  return result.json();
}

// Break up heavy computations
async function heavyComputation(data) {
  for (let i = 0; i < data.length; i++) {
    // Process chunk
    if (i % 1000 === 0) {
      await new Promise(resolve => setTimeout(resolve, 0)); // Yield to
      UI
    }
  }
}
```

Compatibility

Prefer `getContext()` over direct imports

The context API is more stable and less likely to break with SillyTavern updates:

```
// GOOD - Stable API
const { chat, characters, saveSettingsDebounce } = SillyTavern.getContext();

// AVOID - May break with internal changes
import { chat, characters } from '../../../script.js';
```

Use unique module names

Prevent conflicts with other extensions by using a descriptive, unique module name:

```
// GOOD - Specific and unique
const MODULE_NAME = 'my_extension_name';

// BAD - Too generic, likely to conflict
const MODULE_NAME = 'settings';
```

User Experience

Provide clear feedback

Use `toastr` for lightweight notifications and `Popup` for important user interactions. See the [Popups and user feedback](#) section for full details.

For long-running operations, use the **Action loader** instead of blocking the UI silently.

Provide helpful console messages

Use a consistent prefix for your console logs. But do not spam the console with excessive logs in production:

```
const MODULE_NAME = 'MyExtension';

console.log(`${MODULE_NAME} Extension loaded`);
console.debug(`${MODULE_NAME} Processing data:`, data);
console.error(`${MODULE_NAME} Error occurred:`, error);
```

Code Quality

Use bundled libraries from `lib.js`

Before adding new dependencies, check the **Shared libraries** section — SillyTavern bundles many common libraries (lodash, Fuse, DOMPurify, moment, yaml, etc.) that are available via `SillyTavern.libs`.

Initialize settings properly

Always provide defaults and handle missing keys:

```
function loadSettings() {
  // Merge with defaults to handle new keys after updates and initialize if
  // it doesn't exist.
  extensionSettings[MODULE_NAME] = SillyTavern.libs.lodash.merge(
    structuredClone(defaultSettings),
    extensionSettings[MODULE_NAME]
  );
}
```

 [Edit this page](#)